# BountyForge Protocol

Decentralized Developer Bounty Infrastructure on BOT Chain

> ***Abstract:*** *BountyForge is a permissionless, fully on-chain developer bounty marketplace deployed on BOT Chain (Chain ID 677). It resolves the inherent trust deficit in decentralized collaboration by providing a robust smart contract escrow mechanism. Organizations can publish technical requirements and lock reward funds on-chain, while developers can transparently submit work knowing payment is guaranteed upon approval. Featuring support for native BOT and USDT, an integrated dispute resolution process, and pull-based withdrawals, BountyForge creates a secure and efficient ecosystem for Web3 software development.*

## 1. Introduction

The open-source ecosystem and decentralized autonomous organizations (DAOs) rely heavily on distributed talent to build, audit, and maintain software. Historically, coordinating this talent has required informal mechanisms such as community forums, issue trackers, and direct messaging. While these tools facilitate communication, they fail to provide reliable economic guarantees.

BountyForge is designed to eliminate counterparty risk in decentralized software development. By leveraging the immutable properties of BOT Chain, the protocol replaces verbal agreements and centralized custodial platforms with programmatic escrow. This ensures that organizations cannot default on promised rewards, and developers are protected against arbitrary rejections through an on-chain dispute system.

## 2. Market Inefficiencies & Problem Statement

Traditional Web2 and Web3 bounty programs suffer from several systemic inefficiencies that BountyForge addresses directly:

- **Payment Default Risk:** Developers frequently complete work only to face delayed payments, shifting requirements, or outright non-payment because funds were never secured in advance.
- **Opaque Dispute Resolution:** When an organization rejects a submission, developers typically have no recourse. Centralized platforms often side with the funding organization without transparent arbitration.
- **Custodial Intermediaries:** Legacy platforms act as custodians of funds, extracting high fees (often 10–20%) and imposing withdrawal restrictions, minimum limits, and KYC requirements on developers.

## 3. Protocol Architecture

The BountyForge protocol operates entirely through a monolithic Solidity smart contract (`BountyForge.sol`) deployed on BOT Chain. The architecture revolves around two primary user profiles and a strict state machine governing the bounty lifecycle.

### 3.1. Entity Profiles

All participants must register on-chain profiles before interacting with the protocol. These profiles establish identity and track reputation over time.

- **Organizations (Issuers):** Registered via `registerOrg()`. Profiles store metadata such as name, website, and description. Organizations fund bounties and act as primary reviewers.
- **Developers (Contributors):** Registered via `registerDev()`. Profiles log GitHub URLs, technical skills, and historical earnings. Developers submit pull requests (PRs) and repositories as proof of work.

### 3.2. Bounty State Machine

Every bounty instantiated on the platform progresses through a deterministic lifecycle enforced by the smart contract. The table below outlines these states.

| State | Trigger / Condition | Allowed Actions |
|---|---|---|
| **Draft** | Bounty created by Organization, funds not yet deposited. | Edit details, Deposit funds, Cancel. |
| **Open** | Organization deposits total reward (Reward × Max Winners). | Developers can submit work. |
| **InReview** | At least one submission has been made by a Developer. | Organization reviews, approves, or rejects. |
| **Disputed** | Developer raises a dispute within 3 days of rejection. | Admin resolves dispute (Approve/Reject). |
| **Completed** | Approved submissions reach the `maxWinners` threshold. | No further submissions allowed. |
| **Cancelled** | Organization cancels before submissions, or after deadline. | Unused funds refunded to Organization. |

## 4. Economic Model & Incentives

BountyForge is designed to be highly capital-efficient and transparent regarding platform costs. It supports two primary currencies on BOT Chain: native **BOT** (the network gas token) and **USDT** (ERC-20 standard).

### 4.1. Escrow Funding

When an organization transitions a bounty from `Draft` to `Open`, they must deposit the exact total reward required (`rewardPerWinner × maxWinners`). These funds are transferred directly into the BountyForge contract. The contract maps these deposits to the specific bounty ID, ensuring strict segregation of logic.

### 4.2. Platform Fees & Payouts

BountyForge does not charge organizations to deposit funds. Instead, a minimal platform fee is deducted upon successful approval of a submission. This aligns the protocol's success directly with the success of developers getting paid.

- **Fee Rate:** The protocol charges a hardcoded fee of **2.5%** (250 basis points) on all successful payouts.
- **Developer Receipt:** The developer receives exactly **97.5%** of the advertised reward.
- **Fee Accumulation:** Fees are routed into dedicated internal variables (`accumulatedBOTFees` and `accumulatedUSDTFees`) which can only be withdrawn by the platform administrator.

## 5. Dispute Resolution Mechanism

A core differentiator of BountyForge is its on-chain arbitration system. Recognizing that code quality and requirement fulfillment can be subjective, the protocol includes a formalized dispute window.

If an organization rejects a developer's submission via `rejectSubmission()`, a **3-day dispute window** opens. During this period, the developer may call `raiseDispute()` to challenge the decision. This halts the bounty's standard flow and alerts the platform administrator.

The administrator acts as a neutral third-party arbitrator. Importantly, the smart contract enforces **strict limitations on admin power**:

1. The admin can choose to **Approve** the submission (overriding the organization and releasing funds to the developer).
2. The admin can choose to **Confirm Rejection** (siding with the organization and keeping the slot open).
3. **The admin cannot seize, confiscate, or redirect the escrowed funds to themselves or an outside wallet under any circumstances.**

## 6. Security Considerations

The BountyForge protocol was engineered with a security-first mindset, mitigating common attack vectors present in decentralized finance (DeFi) applications.

- **Reentrancy Protection:** A custom `nonReentrant` mutex (`_locked` state variable) is applied to all state-mutating functions that involve native BOT or USDT transfers, preventing recursive call exploits.
- **Pull-Based Withdrawals:** Following Solidity best practices, funds are not "pushed" to developers upon approval. Instead, internal balances (`pendingBOT` and `pendingUSDT`) are credited. Developers must actively call `withdrawBOT()` or `withdrawUSDT()` to pull their funds. This isolates failure domains; a reverting fallback function in a recipient's smart contract wallet cannot halt the protocol's approval logic.
- **Strict Escrow Boundaries:** The `reclaimUnused()` function explicitly checks that the bounty deadline has passed and only refunds the portion of funds that have not already been committed to approved winners.

## 7. Conclusion

BountyForge provides vital infrastructure for the BOT Chain ecosystem, bridging the trust gap between organizations needing talent and developers seeking guaranteed compensation. By eliminating custodial intermediaries, enforcing transparent smart contract escrows, and introducing a fair, restricted-power dispute mechanism, BountyForge sets a new standard for decentralized work coordination. The protocol enables a global, permissionless workforce to build the next generation of decentralized applications with absolute financial security.